Lexical Analysis: The Complete Guide

Lectures 2, 3 and 4 in one document: tokens and regular expressions, every automata method with a worked example, and how a real lexer is built

Compiler Construction Module

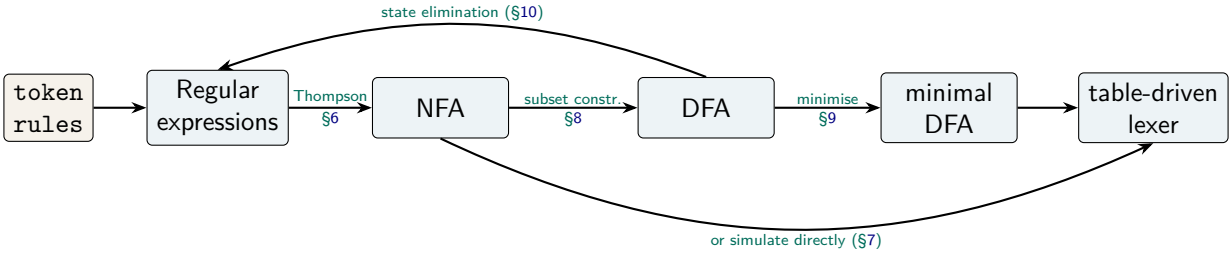
One continuous read, replacing the three separate chapters

How to use this document

This document covers the whole lexical-analysis block of the course in one place.

- **Part I** (Lecture 2, plus the start of Lecture 4) is *the problem*: what a lexer must do, why hand-written lexers break, how to describe tokens with regular expressions, and the two rules that settle conflicts.
- **Part II** is the *automata methods refresher*. Every method you can be asked to perform by hand is written as a recipe, with a worked example. One running example, $(a \mid b)^*abb$, goes through every method, so each result can be checked against the previous one.
- **Part III** (Lectures 3 and 4) turns automata into a working lexer: many tokens at once, longest match with backtracking, costs, lexer states, languages that break the model, and the tools (JFlex, `flex`, a hand-written C scanner).
- **Part IV** is one complete worked example, from a token specification to a scanner run.
- **Part V** is the exam kit: checklists, formulas, traps, and a map from every problem-set question to its section.

Every trace, table and tool output in this document was checked by running it, with a small NFA simulator, `flex` and a C compiler. Regular-expression equivalences were checked on all strings up to length 10.



Concept: Notation and conventions used throughout

- Σ is the alphabet and ε the empty string. For a DFA, $\delta(q, x)$ is the next state and $\emptyset$ the dead (error) state. For an NFA, Δ is the set of edges $q \xrightarrow{x} q'$, where x may be ε.
- $move(S, c)$ and $\varepsilon\text{-}closure(S)$ are defined in §7.
- **Thompson's construction has two conventions for concatenation, and the lectures use both.** *Convention A* (Lecture 3) joins the two parts with one ε-edge. *Convention B*

(Lecture 4, the Dragon book, Problem Set 4) merges the final state of the first part with the initial state of the second. The two give different state counts, so always say which one you use. This is also why Lecture 3 says 10 states for (0 | 1)*1 on one slide and 9 on another (§6).

Summary: What you must be able to do by the end

1. Tokenise a line by hand and give token classes, attributes and line:column positions (§1).
2. Read, write and debug regular expressions for tokens: identifiers, numbers, strings, comments (§3).
3. Apply longest match and rule order correctly (§4).
4. Build Thompson's NFA and count its states and edges under either convention (§6).
5. Simulate an NFA with ε-closures, and run the subset construction (§7, §8).
6. Minimise a DFA by table filling, and know Hopcroft's idea (§9).
7. Turn a DFA back into a regular expression, and use derivatives (§10, §11).
8. Prove that a language needs many DFA states, or is not regular (§12).
9. Build a combined DFA for many tokens, scan with backtracking, and count reads and pushbacks (§13, §14).
10. Explain lexer states, the Python, Fortran and PL/I quirks, and read a `flex` or JFlex specification (§16, §17).

Part I

The Problem

1 What a Lexer Does

Lexing (also called scanning or tokenizing) is the first phase of a compiler. It breaks the input character stream into **tokens**. Tokens fall into a few classes:

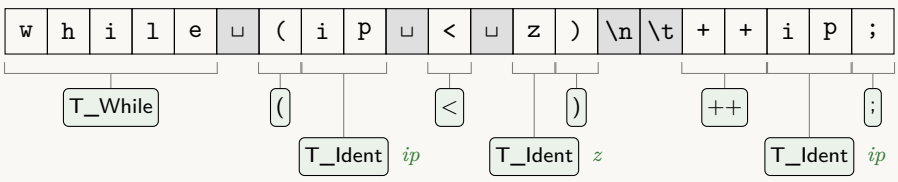
- identifiers, such as `x` and `item_count`;
- keywords, such as `if` and `return`;
- literal constants, such as `23`, `-1.23e-5` and `"Hello!"`;
- operators, such as `+`, `%` and `==`;
- punctuation.

Whitespace and comments are discarded, because later phases never need them. The exception is a tool built to *read* comments, such as Javadoc, which must tokenise the inside of `/** ... */`.

Worked example: Two tokenisations

`if (b == 0) a = "hi"` gives 9 tokens: `if`, `(`, `b`, `==`, `0`, `)`, `a`, `=`, `"hi"`. The blanks are gone, and `==` is one token, not two.

For `while (ip < z)`, a newline, a tab and `++ip;`, the lexer sees one flat stream. The grey cells are discarded:



The lexer had to decide where each token *ends*. It did not stop at `whi`, and it did not split `++` into

two tokens. That decision is the central problem of lexing (§4, §14).

1.1 Token, lexeme, pattern, attribute, position

Concept: The vocabulary

- A **token** is a pair ⟨name, attribute⟩. The *name* (or class) is what the parser sees, and it is a terminal of the grammar. Examples: ⟨Ident, ip⟩, ⟨Operator, <⟩, ⟨“)”, NIL⟩.
- A **lexeme** is the actual text that matched, such as `ip`, `<` or `)`.
- A **pattern** describes all possible lexemes of a token. Patterns are written as regular expressions (§3).
- An **attribute** carries what later phases need. A number token carries its value (`new Number(10.0)`). A string token carries the *interpreted* string: the source `"hello\n"` is 7 characters, but its attribute is the 6-character string with a real newline. An identifier carries its name, or a pointer to its symbol-table entry.
- A **position** (file, line, column) is recorded for each token so errors can be reported as `main.c:42:8: error: ...`.

Token names such as `T_Ident`, `IDENT` or `ID` are invented by the compiler writer: `#define` constants, an `enum`, or a Java `TokenType`.

The interface. As a Java signature, a lexer is `Iterator<Token> lex(Reader input)`. The input is a stream of characters read one at a time. The output is a stream of tokens that the parser *pulls* on demand, one per call (`getNextToken()`, `yylex()`). A lexer also needs a way to report errors, such as an error token or an exception, because not every input can be tokenised.

The four jobs (Lecture 4):

1. cut the stream into lexemes;
2. label each lexeme with a token;
3. record identifiers (and constants) in the symbol table, so every occurrence of `x` gets the *same* pointer;
4. discard blanks and comments.

Why lexing is a separate phase.

- *Simplicity:* the grammar can say *whileStmt* → WHILE (*expr*) *stmt* without mentioning blanks.
- *Efficiency:* the lexer touches every character, so it deserves a specialised machine, a DFA.
- *Portability:* only the scanner deals with encodings, line endings and files.

Choosing tokens. Typically there is one token per keyword, one per punctuation symbol, one for identifiers, several for operators, and one or more for constants. The rule is: *two lexemes get different token names exactly when the grammar must treat them differently.* The six relational operators can share one token `RELOP` with an attribute (LT, LE, ...), since they appear in the same grammar position. `+` and `*` must stay separate, because the grammar encodes their precedence.

2 Why Hand-Written Lexers Break

Hand-written lexing code is a classic source of bugs, including security holes in unsafe languages. The root cause is that **a lexer usually learns where a token ends only by reading one character too many**.

Worked example: Bug 1: `read()` throws away the lookahead

```
while (true) {                          // BROKEN
  int c = input.read();
  if (!isIdChar(c)) break;              // c is lost if it is not an id char
  id.append((char) c);
}
```

On `sum+1`, the loop appends `s`, `u` and `m`, then reads `+`, sees it is not an identifier character, and breaks. The `+` has been consumed and is never seen again. The result is `ID("sum") NUMBER("1")`: the operator has vanished. With `sum + 1` the lost character is a harmless blank, which is why such bugs hide. The fix is to **peek before consuming**: `while (isIdChar(input.peek())) id.append(input.read());`.

Worked example: Bug 2: committing too early

A lexer that emits `ASSIGN` as soon as it reads `=` turns `x==y` into `ID(x) ASSIGN ASSIGN ID(y)` instead of `ID(x) EQ ID(y)`. The fix is the same: after `=`, peek. If the next character is also `=`, consume both and emit `EQ`. A lexer generator avoids the problem entirely: given rules for `"="` and `"=="`, longest match always prefers `"=="`.

Worked example: One character of lookahead is not always enough

Take the specification `abc | (abc)*d` and the input `abcabcx`. After `abc` a token is available, but `(abc)*d` is still possible, so the lexer keeps reading: `abc`, then `x`. Only then does `(abc)*d` fail, and the correct answer, the token `abc`, lies 4 characters back. Recovering it needs **backtracking** (putting several characters back), not just one-character **lookahead**. The scan-and-backtrack algorithm of §14 does exactly this.

Positions are tracked while reading. Keep a column counter that increases on every character and a line counter that increases on every newline, where the column resets to 1. In

```
1 x = 1
2 y = 2
3 z = x +
```

`y` is at 2:1, and the final `+` is at 3:7 (columns: `z`=1, blank=2, `=`=3, blank=4, `x`=5, blank=6, `+`=7). Computing positions afterwards would mean re-reading the input.

3 Describing Tokens: Regular Expressions

Instead of hand-writing lexing code, we *declare* the tokens with regular expressions and let a tool synthesise the lexer. That is easier to get right, and the result is efficient and safer.

An **alphabet** Σ is a finite set of characters. A **string** is a finite sequence of characters. A **language** is a set of strings. A regular expression R denotes a language $L(R)$.

Concept: The five core constructs

Syntax	Language	Name
a	$\{\text{“}a\text{”}\}$	one character
ε	$\{\text{“”}\}$	the empty string
$R \mid S$	$L(R) \cup L(S)$	union (alternation)
$R\,S$	$\{xy : x \in L(R),\ y \in L(S)\}$	concatenation
R^*	$L(\varepsilon) \cup L(R) \cup L(RR) \cup L(RRR) \cup \cdots$	Kleene star

Parentheses only group. **Precedence**, from strongest to weakest: star, then concatenation, then union. So $ab^* \mid c$ means $(a(b^*)) \mid c$. The **regular languages** are exactly the languages these five constructs can describe.

Worked example: Reading regular expressions

Expression	Language	Comment
$a\varepsilon$	$\{a\}$	ε is the identity for concatenation
$a \mid b$	$\{a, b\}$	
$(ab)^*$	$\{\varepsilon, ab, abab, \dots\}$	
$(a \mid \varepsilon)b$	$\{ab, b\}$	“optionally a, then b”: this is $a?b$
$(a \mid b)^*abb$	strings over $\{a, b\}$ ending in abb	**our running example**
$(+ \mid - \mid \varepsilon)$[0-9]$^*(0 \mid 2 \mid 4 \mid 6 \mid 8)$	even numbers, optional sign	

Method for testing a string: split the expression into its concatenated parts, and try to assign each character to a part.

- For $(a \mid b)^*abb$: `abb`, `babb` and `aabb` are in. `ab` is out (it does not end in `abb`), and so is `abbb` (it ends in `bbb`).
- For the even numbers: `-24` and `0` are in, and so is `0034`, because leading zeros are allowed. `7` is out (odd last digit), and so is `+` (no digit).

Concept: Shorthands (no extra power) and abbreviations

Shorthand	Means	Shorthand	Means
$R?$	$R \mid \varepsilon$	`[abc]`	$a \mid b \mid c$
R^+	RR^*	`[a-zA-Z]`	any letter
`[^ab]`	any character but a, b	`.`	any character except newline
$R\{n, m\}$	n to m copies of R	`"x*"`	the literal text `x*`

Abbreviations (also called regular definitions) name pieces of a pattern: `digit = [0-9]`, then `posint = [1-9]{digit}*`, which expands to `[1-9][0-9]*`. They must **not be recursive**. Something like `x = {x}a | ε` never finishes expanding. More fundamentally, recursion would give the power of context-free grammars (Chomsky type 2) instead of regular languages (type 3). That power belongs to the parser.

Method: Writing a token regex

1. List what must be accepted and what must be rejected, including the edge cases.
2. Name the pieces with abbreviations.
3. Concatenate the pieces, using ? for optional parts and * or $^+$ for repeated ones.

4. Test it on your accept and reject lists.

Worked example: Worked token patterns

- **Identifiers:** `[a-zA-Z_][a-zA-Z0-9_]*`. The first character is never a digit, so a scanner can tell a name from a number by its first character.
- **Integers** (JFlex, Lecture 2): `"0" | "-"?[1-9]{Digit}*`, which means no leading zeros.
- **Scientific notation:** `digit = [0-9]`, `decimal = digit+ "." digit+`, and then `sci = decimal ("e"|"E") ("+"|"-")? digit+`. This matches `2.5e10`, `3.0E-4` and `42.0e0`.
- **Hex literals:** `hexlit = "0" [xX] {hexdigit}+`, with `hexdigit = [0-9a-fA-F]`. It matches `0x1A3F`, `0xff` and `0X1A`. It rejects `0x` (no digit), `x1F` (no `0`) and `0x1G` (`G` is not a hex digit; the lexer would take `0x1` and stop).
- **Floats** in C style (`2.0`, `.1e0`, `20.e99`):
  `[+-]?(([0-9]+(\.[0-9]*)?|\.[0-9]+)([eE][+-]?[0-9]+)?)`
  The mantissa is either digits with an optional point and fraction, or a point and digits. The exponent is optional. This pattern *also* matches `17`, so it overlaps with integers, and rule order must decide (§4).
- **Strings with escapes** (`"he said \"hi\""`): `"\"" ( [^"\\] | "\\" . )* "\""`. Between the quotes there are any number of either ordinary characters (not a quote and not a backslash), or a backslash followed by any character. Escapes are needed twice: once for the language being lexed, and once for the specification syntax.
- **Block comments** are the hardest classic. Their pattern is derived from a DFA in §17.

Warning / Gotcha: Tokens are what the spec says, not what looks natural

With Lecture 2's rules, where `Float = digit+ "." digit+` has no sign, `return -3.5 + count_2;` gives `return` (keyword), `-` (operator), `3.5` (float), `+`, `count_2` (identifier) and `;`. "Negative 3.5" is the parser's business. The opposite mistake is putting a sign *inside* a number pattern. With `INT = [+-]?[0-9]+`, the input `x-1` becomes `ID(x) INT(-1)`, because `-1` is longer than `-`, and the parser sees two operands in a row.

4 Resolving Conflicts: Two Rules and a Safety Net

A specification is a list of rules $R_1, \ldots, R_n$, each with an action. Several rules can match at the same position, and they can match different lengths. Two rules decide, *in this order*:

Concept: The tie-breaking rules

1. **Longest match** (maximal munch): among all rules matching a prefix of the remaining input, keep the match that consumes the *most* characters.
2. **Rule order** (priority): only if several rules match that same longest prefix, the rule written *first* wins.

Safety net: a **catch-all** rule (`.` or Σ) placed *last* matches any single character and reports an error. It never steals a real token: any other rule that matches 2 or more characters wins on length, and any rule that matches exactly 1 wins the tie by order.

Worked example: Applying the two rules

Rules	Input	Competing matches	Result
`"in"`, identifier	`infinity`	2 vs. 8 characters	`ID(infinity)` by longest match
`"if"`, identifier	`if`	2 vs. 2	`IF`, by order (keyword written first)
identifier, `"if"`	`if`	2 vs. 2	`ID(if)`: the keyword is lost
`"class"`, identifier	`classroom`	5 vs. 9	`ID(classroom)`, never CLASS + ID(room)
`"end"`, identifier	`ending`	3 vs. 6	`ID(ending)`
`"<"`, `"<="`	`<=`	1 vs. 2	`<=`; order is irrelevant

Rule order is consulted *only* when two rules match the same longest lexeme. That is why keywords are written as exact literal strings placed before the identifier rule: longest match protects `infinity`, and order handles `in` itself.

Worked example: Why longest match is the right default

Rules: `IF = if`, `EQ = =`, `NUM = [0-9]+`, `ID = [a-z][a-z0-9]*`. The input `ifu26 = 60` could be cut as `IF ID(u26) EQ NUM(60)`, or as `ID(ifu26) EQ NUM(60)`, or as `ID(ifu) NUM(26) EQ NUM(6) NUM(0)`, and more. Longest match picks exactly one cut, `ID(ifu26) EQ NUM(60)`, which is what every programmer means.

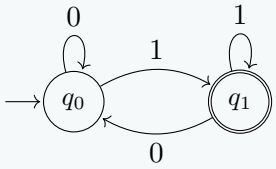
Part II

Automata Methods Refresher

5 Finite Automata in One Page

Concept: DFA

A **deterministic finite automaton** is $M = (Q, \Sigma, \delta, q_0, F)$: a finite set of states Q, an alphabet Σ, a transition *function* $\delta : Q \times \Sigma \to Q$, a start state q_0 and accept states F. It reads one symbol at a time, and it accepts if it ends in F. Missing transitions go to a non-accepting **dead state** $\emptyset$, which loops to itself, so δ is total. The transition table *is* the implementation:



δ	0	1
q_0	q_0	q_1
q_1	q_0	q_1

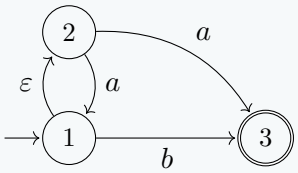
```
q = q0;
for (i = 0; i < n; i++)
  q = delta[q][input[i]];
return q in F;
```

This DFA accepts $(0 \mid 1)^*1$, the binary strings ending in 1. Running a DFA costs one table lookup per character, $O(n)$.

Concept: NFA

A **nondeterministic finite automaton** $N = (Q, \Sigma, \Delta, s_0, F)$ replaces δ by a *relation* $\Delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$. A state may have zero, one or several edges on the same symbol, and ε-edges that read nothing. A string is accepted if *some* path spells it and ends in F. To **reject**, you must

show that *every* path fails; one failed path proves nothing.



	a	b	ε
1	$\emptyset$	$\{3\}$	$\{2\}$
2	$\{1,3\}$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

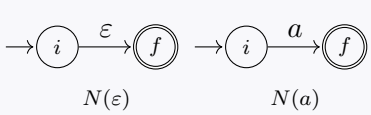
Table entries are *sets*. Each trip $1 \xrightarrow{\varepsilon} 2 \xrightarrow{a} 1$ reads one a, and the machine exits by $1 \xrightarrow{b} 3$ or $2 \xrightarrow{a} 3$. So $L = L(a^*b \mid a^+)$. On `ba`, no path survives, because state 3 has no outgoing edge.

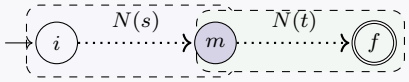
Equivalence (Kleene). Regular expressions, NFAs and DFAs describe exactly the same languages. The methods below convert between them: regex → NFA (Method 1), NFA → DFA (Method 3), DFA → regex (Method 5).

6 Method 1: Regex to NFA (Thompson's Construction)

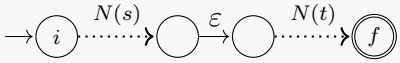
Build the NFA by induction on the structure of the regular expression. Every fragment has exactly one start state and one accept state, so fragments snap together.

Mechanism: The rules

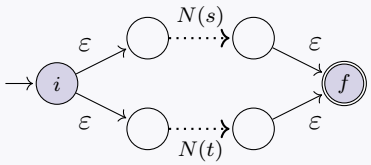




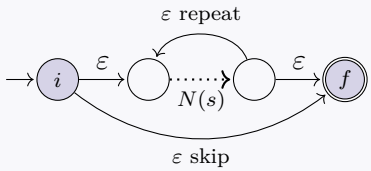
$N(st)$, **convention B** (Lecture 4, Problem Set 4): the final state of $N(s)$ *is* the initial state of $N(t)$; nothing is added



$N(st)$, **convention A** (Lecture 3): one ε-edge joins the parts; no new state



$N(s \mid t)$: new i and f, four ε-edges



$N(s^*)$: new i and f, four ε-edges

Dotted arrows stand for a whole sub-automaton, from its start to its accept state. New states are shaded.

Method: Thompson by hand

1. **Parse first.** Write the expression as a tree, using precedence (star, then concatenation, then union). Parentheses are not operators.
2. Build **inside out**: leaves first, then each operator from its children.
3. **Count to check** (below), and check properties (P1) to (P3).

Proposition 6.1 (Exact counts)**.** *Let r have n_ℓ leaves (symbols or ε), $n_|$ unions, n_* stars and $n_.$ concatenations.*

	States $\lvert Q\rvert$	*Edges* $\lvert E\rvert$
Convention A (ε-edge)	$2(n_\ell + n_\mid + n_*)$	$n_\ell + 4(n_\mid + n_*) + n.$
Convention B (merge)	$2(n_\ell + n_\mid + n_*) - n.$	$n_\ell + 4(n_\mid + n_*)$

Proof. A leaf creates 2 states and 1 edge. A union or star creates 2 states and 4 edges. A concatenation adds 1 edge (convention A), or merges two states into one (convention B). □

Proposition 6.2 (Properties of Thompson NFAs, slide 63 of Lecture 4)*(P1) There are at most* $2 \times$ (*#operators+#operands*) *states. Under convention B the exact count is* 2(*#ops+#operands*)$-3n.$*, so the bound is tight only when there is no concatenation.*

(P2) There is one start state with no incoming edge, and one accept state with no outgoing edge.

(P3) Every non-accepting state has either exactly one outgoing edge, or two outgoing edges that are both ε*. So there are at most* $2|Q|$ *edges.*

(P2) and (P3) hold for the base cases, and every rule preserves them. (P2) is what makes merging safe. Suppose you built a^* and b^* "lazily" as single states that are both start and accept with a loop, then merged them for a^*b^*. The result would be one state with loops on a and b, and it would wrongly accept `ba`.

Worked example: Lecture 3's example: $(0 \mid 1)^*1$, convention A

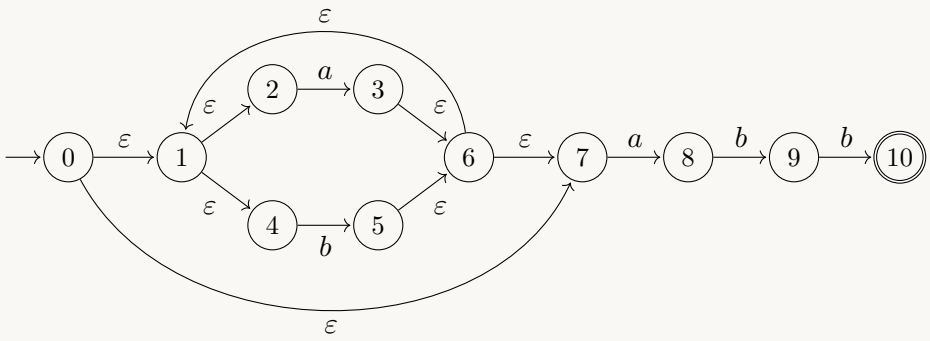
The tree is $((0 \mid 1)^*) \cdot 1$. In build order:

Rule	New states	Total
symbol 0	B, C	2
symbol 1	D, E	4
union 0 | 1	A (start), F (accept)	6
star	S (start), G (accept)	8
symbol 1	H, J	10
concatenation	none (edge $G \xrightarrow{\varepsilon} H$)	10

The edges are: $S \to A$, $S \to G$ (skip), $A \to B$, $A \to D$, $B \xrightarrow{0} C$, $D \xrightarrow{1} E$, $C \to F$, $E \to F$, $F \to G$, $F \to A$ (repeat), $G \to H$, $H \xrightarrow{1} J$. All unlabelled edges are ε. Under convention B, G and H merge, which gives the **9 states** that Lecture 3's concept check mentions. Problem Set 3's $(a \mid b)^*c$ has the same tree shape, hence the same counts.

Worked example: Running example: $(a \mid b)^*abb$, convention B

The tree is $((((a \mid b)^* \cdot a) \cdot b) \cdot b)$: 5 leaves, 1 union, 1 star and 3 concatenations. So $|Q| = 2 \cdot 7 - 3 = 11$ and $|E| = 5 + 8 = 13$. Convention A would give 14 states and 16 edges.



States 7, 8 and 9 are the three merge points. (P3) checks out: states 0, 1 and 6 have two ε-edges each, and every other non-final state has exactly one edge.

7 Method 2: Simulating an NFA

Do not guess a path: keep the *set* of all states the NFA could be in.

- $move(S, c)$ is the set of states reachable from some state of S by one edge labelled c.
- ε-$closure(T)$ is the set of states reachable from T by *zero* or more ε-edges. It always contains T. Compute it as a graph search: put T on a stack; pop a state; add every ε-successor not yet seen; repeat until the stack is empty.

Method: NFA simulation (Lecture 4, slide 64)

1 $S \leftarrow \varepsilon\text{-}closure(\{s_0\})$;
2 **foreach** *input character* c **do**
3 $\quad S \leftarrow \varepsilon\text{-}closure(move(S, c))$;
4 accept iff $S \cap F \neq \emptyset$;

After reading a prefix w, S is exactly the set of states reachable by a path spelling w (induction on the length of w).

Worked example: Running example: `abb` and `ab`

Read	*move*	S after closure	Notes
(start)		$\{0, 1, 2, 4, 7\}$	from 0: ε to 1 and 7; from 1 to 2 and 4
`a`	$\{3, 8\}$	$\{1, 2, 3, 4, 6, 7, 8\}$	from 3: to 6, then 7 and 1, then 2 and 4
`b`	$\{5, 9\}$	$\{1, 2, 4, 5, 6, 7, 9\}$	
`b`	$\{5, 10\}$	$\{1, 2, 4, 5, 6, 7, 10\}$	contains 10: **accept**

On `ab` the run stops at $\{1, 2, 4, 5, 6, 7, 9\}$. That set has no 10, so `ab` is rejected. Keep these sets: Method 3 rebuilds exactly them.

Proposition 7.1 (Cost)**.** *On an input of length* $|S|$*:*

- *an arbitrary NFA costs* $O(|S|\,|Q|^2)$ *(Lecture 4, slide 73);*
- *a Thompson NFA costs* $O(|S|\,|Q|)$*;*
- *a DFA costs* $O(|S|)$*.*

Proof. A step scans the outgoing edges of each state in the current set. With a Boolean array for membership, each state is handled at most once per step. In general, $|Q|$ states may each have $|Q|$ successors, so a step scans $\Theta(|Q|^2)$ edges: **the square comes from the number of edges**. By (P3), a Thompson state has at most 2 edges, so a step costs $O(|Q|)$. A DFA step is one lookup. □

Key insight. NFA simulation is the subset construction done lazily: one character at a time, and only for the sets the input actually visits.

8 Method 3: NFA to DFA (Subset Construction)

Precompute the simulation for *every* possible input. Each DFA state is a set of NFA states.

Method: Subset construction by hand

1. The start state is $\varepsilon\text{-}closure(\{s_0\})$. Name it A and put it in a worklist.
2. Take an unprocessed set S. For each symbol c, compute $\varepsilon\text{-}closure(move(S, c))$. If the result is new, name it and add it to the worklist. If it is empty, it is the dead state (usually not drawn).
3. Repeat until no new set appears.
4. A DFA state is **accepting** if it contains an NFA accepting state.

In formulas: $\delta(S, c) = \varepsilon\text{-}closure(move(S, c))$. Only the reachable sets are built.

Worked example: Running example: the DFA for $(a \mid b)^*abb$

DFA	NFA set	on a	on b	Accept?
A	$\{0, 1, 2, 4, 7\}$	$move = \{3, 8\} \Rightarrow$ B	$\{5\} \Rightarrow$ C	
B	$\{1, 2, 3, 4, 6, 7, 8\}$	$\{3, 8\} \Rightarrow$ B	$\{5, 9\} \Rightarrow$ D	
C	$\{1, 2, 4, 5, 6, 7\}$	$\{3, 8\} \Rightarrow$ B	$\{5\} \Rightarrow$ C	
D	$\{1, 2, 4, 5, 6, 7, 9\}$	$\{3, 8\} \Rightarrow$ B	$\{5, 10\} \Rightarrow$ E	
E	$\{1, 2, 4, 5, 6, 7, 10\}$	$\{3, 8\} \Rightarrow$ B	$\{5\} \Rightarrow$ C	yes (contains 10)

The sets A, B, D and E are exactly the sets of the simulation of `abb` in Method 2. There are 5 DFA states for an 11-state NFA. Each state remembers "how much of `abb` I have just seen": A and C have seen nothing useful, B has seen `a`, D `ab`, and E `abb`.

Lecture 3's version, on the convention-A NFA for $(0 \mid 1)^*1$: $Q_0 = \varepsilon\text{-}closure(\{S\}) = \{S, A, B, D, G, H\}$. On 0 it goes to $Q_1 = \{C, F, A, B, D, G, H\}$, and on 1 to $Q_2 = \{E, F, A, B, D, G, H, J\}$, which is accepting because it contains J. That gives 3 DFA states.

Warning / Gotcha: The blow-up is real

A DFA state is a subset of Q, so there can be up to $2^{|Q|}$ of them. For $L_k = (a \mid b)^*a(a \mid b)^{k-1}$ (the k-th symbol from the end is a), an NFA has $k + 1$ states, but every DFA has at least 2^k (proof in §12). This is why the pipeline keeps the NFA stage (Lecture 3, concept check 1): Thompson's step is simple and linear, and the step that can explode is isolated in the subset construction.

9 Method 4: DFA Minimisation

Two states are **equivalent** if, from either one, exactly the same remaining suffixes are accepted. Otherwise they are **distinguishable**, and a suffix that one accepts and the other rejects is a *witness*.

By the Myhill–Nerode theorem, merging equivalent states gives the **unique** minimal DFA, up to renaming.

Method: Table filling (Lecture 3)

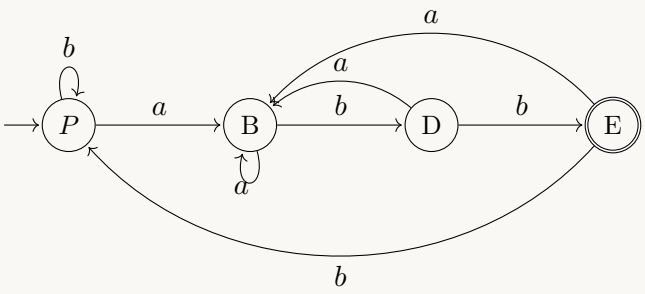
1. List all pairs of states (an n-state DFA has $\binom{n}{2}$ pairs). Include the dead state if the DFA is incomplete.
2. **Rule 1:** mark every pair made of one accepting and one non-accepting state. The witness is ε.
3. **Rule 2**, in sweeps: mark (p, q) if some symbol x sends them to a pair $(\delta(p, x), \delta(q, x))$ that is *already* marked. The witness is x followed by that pair's witness. Repeat full sweeps until one sweep marks nothing new.
4. Unmarked pairs are equivalent. **Merge** each class into one state, and **relabel every transition** that pointed at a merged state, wherever it comes from. The new start state is the class of the old start, and a class is accepting if its states were.

Worked example: Running example: minimising A–E

There are $\binom{5}{2} = 10$ pairs. Each sweep uses only the marks from earlier sweeps.

Pair	Marked when	Why
(A,E), (B,E), (C,E), (D,E)	Rule 1	E accepts, the others do not (witness ε)
(A,D)	sweep 1	on b: (C,E) is marked; witness b
(B,D)	sweep 1	on b: (D,E) is marked; witness b
(C,D)	sweep 1	on b: (C,E) is marked; witness b
(A,B)	sweep 2	on b: (C,D) was marked in sweep 1; witness bb
(B,C)	sweep 2	on b: (D,C) was marked in sweep 1; witness bb
(A,C)	never	on a: (B,B); on b: (C,C). Same targets, so nothing can mark it

Sweep 3 marks nothing, so we stop. **A** $\equiv$ **C**: merge them into P, and relabel every arrow into C as an arrow into P.



	a	b
$P = \{A, C\}$ (start)	B	P
B	B	D
D	B	E
E (accept)	B	P

The 4 states mean "seen nothing useful", "seen a", "seen ab" and "seen abb". §12 proves that no DFA can have fewer.

Lecture 3's example, Q_0, Q_1, Q_2: Rule 1 marks (Q_0, Q_2) and (Q_1, Q_2). Q_0 and Q_1 go to the same states on 0 and on 1, so Rule 2 never fires, and they merge into the 2-state DFA shown in §5.

Concept: Hopcroft's algorithm (Lecture 4): the same result in $O(n \log n)$

Table filling looks at every pair, which costs $O(n^2)$. Hopcroft works with *blocks* instead.

1. Start from the partition $\{F,\ Q \setminus F\}$.

2. Split a block whenever its states disagree on which block some symbol leads to.
3. Use only the smaller half of each split for further splitting. Then each state is charged at most $\log_2 n$ times.

On A–E: start with $\{E\}$ and $\{A, B, C, D\}$. On b, only D enters $\{E\}$, which splits off $\{D\}$. On b, only B enters $\{D\}$, which splits off $\{B\}$. Nothing splits $\{A, C\}$. The result is the same 4 blocks.

10 Method 5: DFA to Regex (State Elimination)

This is the "Kleene construction" arrow of Lecture 4. It proves that every DFA has a regular expression.

Method: State elimination

1. Add a new start state with an ε-edge to the old start, if the old start has incoming edges. Add a new final state f, with ε-edges from every old accepting state.
2. Eliminate the other states one at a time. To remove q, with self-loop L (or none), replace every path $p \xrightarrow{A} q \xrightarrow{B} r$ by an edge $p \xrightarrow{AL^*B} r$. Join it with $\mid$ to any existing $p \to r$ edge.
3. When only the start state and f remain, read off the expression. If the start has a loop X and the edge to f is Y, the answer is X^*Y.

Worked example: Running example: back from the minimal DFA

Here the start state P is kept, since its loop is handled at the end, and f is added with $E \xrightarrow{\varepsilon} f$.

1. **Eliminate E** (no loop). It is entered from D on b and leaves to B on a, to P on b, and to f on ε. New edges: $D \xrightarrow{ba} B$, $D \xrightarrow{bb} P$, $D \xrightarrow{b} f$.
2. **Eliminate D** (no loop). It is entered from B on b; D's edges are $a \mid ba$ to B, bb to P, and b to f. New edges: $B \xrightarrow{b(a|ba)} B$, which joins the loop a to give B the loop $a \mid ba \mid bba$. Also $B \xrightarrow{bbb} P$ and $B \xrightarrow{bb} f$.
3. **Eliminate B** (loop $M = a \mid ba \mid bba$). It is entered from P on a. New edges: $P \xrightarrow{aM^*bbb} P$, which joins the loop b, and $P \xrightarrow{aM^*bb} f$.

$$L = (b \mid a(a \mid ba \mid bba)^*bbb)^*\ a(a \mid ba \mid bba)^*bb.$$

This is much uglier than $(a \mid b)^*abb$, but it is the same language (checked on all strings up to length 10). Different elimination orders give different, equivalent expressions. On the 2-state DFA of §5, adding f and eliminating q_1 (loop 1) gives $(0 \mid 11^*0)^*11^*$, which equals $(0 \mid 1)^*1$.

11 Method 6: Brzozowski Derivatives

This is a third way to match (Lecture 3, slide 33): differentiate the *expression*. The derivative $\partial_a R$ describes what is left of $L(R)$ after reading a. Let $\nu(R) = \varepsilon$ if R accepts the empty string, and $\nu(R) = \emptyset$ otherwise.

$$\begin{array}{lll} \partial_a a = \varepsilon, & \partial_a b = \emptyset\ (b \neq a), & \partial_a \varepsilon = \partial_a \emptyset = \emptyset, \\ \partial_a(R \mid S) = \partial_a R \mid \partial_a S, & \partial_a(R^*) = (\partial_a R)\, R^*, & \partial_a(RS) = (\partial_a R)\, S \ \mid\ \nu(R)\, \partial_a S. \end{array}$$

The slide omits the concatenation rule and ν. Without them, nothing with a concatenation can be differentiated. A string $w = a_1 \cdots a_n$ matches R if and only if $\partial_{a_n} \cdots \partial_{a_1} R$ is nullable.

Worked example: Derivatives build the minimal DFA directly

Let $R = (0 \mid 1)^*1$.

- $\partial_0 R = ((\varepsilon \mid \emptyset)(0 \mid 1)^*)1 \mid \varepsilon\,\emptyset \equiv R$.
- $\partial_1 R = ((\emptyset \mid \varepsilon)(0 \mid 1)^*)1 \mid \varepsilon\,\varepsilon \equiv R \mid \varepsilon$.
- From $R \mid \varepsilon$: ∂_0 gives R again, and ∂_1 gives $R \mid \varepsilon$ again.

So there are two expressions, R (not nullable) and $R \mid \varepsilon$ (nullable): exactly the 2-state DFA, with no NFA and no minimisation. The catch is that you must simplify ($\varepsilon R \equiv R$, $\emptyset R \equiv \emptyset$, $R \mid \emptyset \equiv R$) to recognise repeated states.

12 Method 7: Proving a DFA Needs Many States, or Is Impossible

Method: Distinguishable strings

Find strings $w_1, \ldots, w_k$ such that for every pair $w_i \neq w_j$ some suffix z puts exactly one of $w_i z$, $w_j z$ in the language. Then every DFA needs at least k states: a DFA in the same state after w_i and w_j would treat $w_i z$ and $w_j z$ alike. If you can find *infinitely* many such strings, the language is **not regular**.

Worked example: Three applications

- **The running example needs 4 states.** Take ε, a, ab and abb. The suffix bb separates a from ε (abb is in, bb is out). The suffix b separates ab from both ε and a. And abb is the only one already in the language. So 4 is optimal, confirming Method 4.
- **L_k needs 2^k states.** Let $u \neq v$ have length k and differ at position i, with $u_i = a$ and $v_i = b$. Append b^{i-1}. Now the k-th symbol from the end is position i, so $ub^{i-1} \in L_k$ and $vb^{i-1} \notin L_k$. All 2^k strings of length k are pairwise distinguishable. Minimisation cannot help, since the bound holds for every DFA.
- **Nesting is not regular.** The strings $(^1, (^2, (^3, \ldots$ are pairwise distinguishable by the matching number of). So balanced parentheses, nested comments (`/* /* */ */`) and Python indentation (§16) need a counter or a stack. Equivalently, by pigeonhole: a machine with N states cannot tell $N + 1$ nesting depths apart.

Method: The pumping lemma, when the question asks for it by name

Every regular language L has a constant p (the number of states of some DFA for it) such that any $w \in L$ with $|w| \geq p$ splits as $w = uvz$ with $|uv| \leq p$, $|v| \geq 1$ and $uv^i z \in L$ for every $i \geq 0$. Reason: reading the first p symbols of w visits $p + 1$ states, so some state repeats; v is the loop between the two visits, and a loop can be skipped or repeated without changing where the machine ends up.

To prove that L is *not* regular: suppose it is, with constant p; pick one $w \in L$, $|w| \geq p$, whose first p symbols are all alike; then every allowed split has v inside that block, and pumping ($i = 0$ or $i = 2$) produces a string outside L. Example, $L = \{a^n b^{2n}\}$ (grammar $S \to aSbb \mid \varepsilon$): take $w = a^p b^{2p}$; any split has $v = a^j$ with $j \geq 1$; then $uv^2 z = a^{p+j} b^{2p}$, which would need $2(p + j)$ letters b to be in L and has only $2p$. Contradiction, so L is not regular. The distinguishable-strings argument above gives the same result in one line ($a^1, a^2, \ldots$ are separated by the suffixes $b^2, b^4, \ldots$). Use whichever the question names; when it names neither, distinguishable strings is shorter and also gives the state count.

Part III

From Automata to a Lexer

13 Many Tokens in One Automaton

A specification is one big alternation $R_1 \mid R_2 \mid \cdots \mid R_n$, but the lexer must also know *which* R_i matched. There are two ways to handle many tokens (Lecture 4, slide 65): simulate each token's NFA separately, or **merge** them. Everyone merges.

Method: Building the lexer automaton

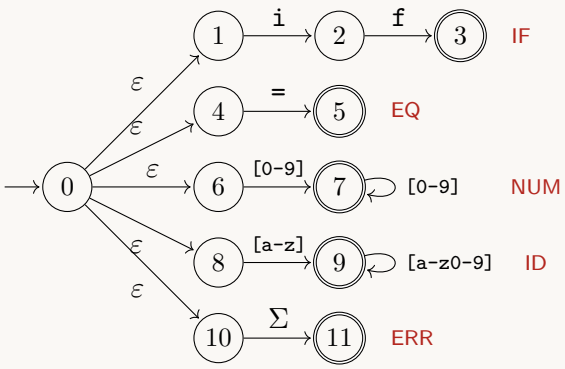
1. Build an NFA for each rule (Method 1).
2. Add a new start state with an ε-edge to each rule's start. Keep the accept states **separate**, and label each with its token.
3. Run the subset construction (Method 3). A DFA state that contains accept states of several tokens takes the label of the **highest-priority** one, meaning the rule written first.
4. Scan with longest match (§14). Labels are consulted only once the length is fixed.

Worked example: Lecture 3: the `if` / identifier collision

Two branches: $K_0 \xrightarrow{i} K_1 \xrightarrow{f} K_2$ (IF), and $D_0 \xrightarrow{\text{letter}} D_1$, where D_1 loops on letters, digits and `_` (ID). After reading `i`, the active states are $\{K_1, D_1\}$, and D_1 accepts as ID. After `if`, they are $\{K_2, D_1\}$, and *both* accept. The DFA state $\{K_2, D_1\}$ gets the label IF, because `"if"` is written before `{Identifier}`. On `infinity` this state is passed, but the scan continues and ends at ID with 8 characters: longest match first.

Worked example: Lecture 4: four tokens, a catch-all, and a full trace

Rules, in priority order: IF `if`, EQ `=`, NUM `[0-9]+`, ID `[a-z][a-z0-9]*`, and a catch-all ERR (any character Σ).



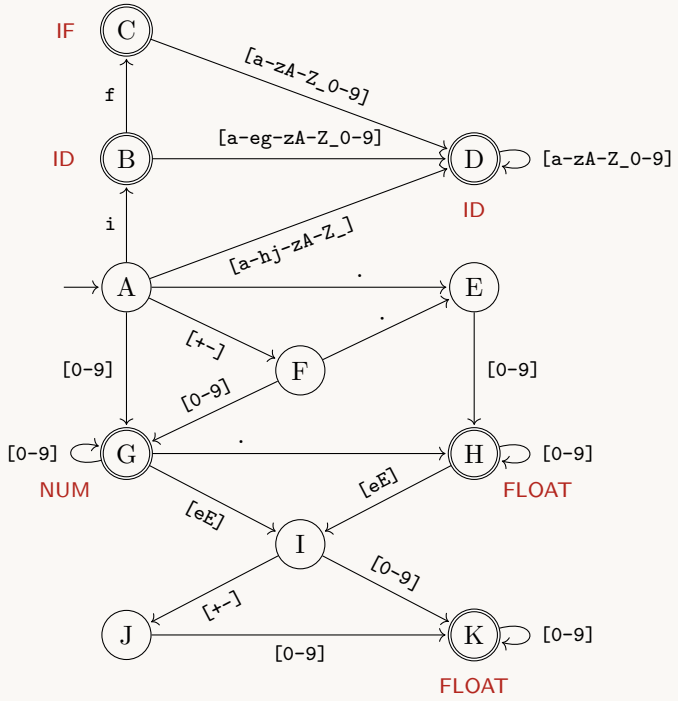
Trace of `ifu26=60`:

Read	S	Last match
`i`	$\{2, 9, 11\}$	ID (1); ID beats ERR
`f`	$\{3, 9\}$	IF (2); IF beats ID
`u,2,6`	$\{9\}$	ID (5)
`=`	$\emptyset$	emit **ID ifu26**, push back `=`
`=`	$\{5, 11\}$	EQ (1)
`6`	$\emptyset$	emit **EQ**, push back `6`
`6,0`	$\{7\}$	end of input: emit **NUM 60**

That is 10 reads for 8 characters. The row for `f` is the lesson: after 2 characters the best label is IF, but the scan goes on, and the longer ID wins. On `x = 60`, with blanks skipped, the lexer outputs $\langle \text{ID}, x\rangle\langle \text{EQ}\rangle\langle \text{NUM}, 60\rangle$.

Worked example: Lecture 4: the combined DFA for IF, ID, NUM, FLOAT

The rules, in priority order, are IF, ID `[a-zA-Z_][a-zA-Z_0-9]*`, NUM `[+-]?[0-9]+`, and FLOAT (the C-style float of §3). The subset construction on the merged NFA gives the following states. NFA numbering follows slide 79: IF is 2–4, ID 5–6, NUM 7–9, FLOAT 10–18.



State	NFA set	Label
A	$\{1, 2, 5, 7, 8, 10, 11\}$	—
B	$\{3, 6\}$	ID
C	$\{4, 6\}$	IF†
D	$\{6\}$	ID
E	$\{14\}$	—
F	$\{8, 11\}$	—
G	$\{8, 9, 12\}$	NUM†
H	$\{14, 15\}$, $\{13\}$	FLOAT
I	$\{16, 17\}$	—
J	$\{17\}$	—
K	$\{17, 18\}$	FLOAT

† Won by priority: C also holds ID's accept state, and G holds FLOAT's, since FLOAT matches `17` too. The construction gives 12 states; the slide's H merges the equivalent $\{13\}$ and $\{14, 15\}$.

Slide 80's exercise, with a blank rule and a catch-all assumed:

- `if 17`: A→B→C (IF), then stuck on the blank, so emit **IF**. The blank is skipped. Then A→G→G, end of input: emit **NUM 17**. That is 7 reads.
- `3e-y`: A→G (NUM), then e to I, $-$ to J, and stuck on y. Back up to G and emit **NUM 3**, pushing back 3 characters (`e-y`). Then `e` gives **ID e**. `-` leads to F, which never accepts, so the catch-all reports **an error on -**. Finally **ID y**. That is 9 reads.

14 Longest Match in a Running Lexer

A DFA answers "is this whole string a token?". A lexer needs "where does the longest token starting here end?". It must not stop at the first accepting state (`in` versus `infinity`), and it must not trust the point where it gets stuck either (`abcabx` with `abc | (abc)*d`).

Method: Scan and backtrack (Lecture 3)

```
start = i; q = q0;
// Phase 1: read ahead until stuck
while (true) {
  input[i] = read();
  if (input[i] == EOF || delta(q,input[i]) ==
     DEAD) break;
  if (q in F) clear the stack;
  push q;
  q = delta(q, input[i]); i = i + 1;
}
// Phase 2: back up to the last final state
while (q not in F) {
  if (stack is empty) fail;
  q = pop(); unread(input[i]); i = i - 1;
}
return input[start .. i-1];
```

Phase 1 keeps going past accepting states, in case a longer match comes. It stops only at end of input or at the dead state.
The stack is cleared whenever an accepting state is left. So it holds only the states visited *since the last accept*.
Phase 2 pops and unreads one character per state until it lands on an accepting state, which is the longest match. If the stack runs out, no token starts here.
The NFA version (Lecture 4) is the same idea: remember the label and end position of the last time the state set contained an accepting state. When the set becomes empty, emit that match and push back everything read after it.

Worked example: Trace: rules `abc` and `(abc)*d`, input `abcabx`

Read	State after	Accepting?	Stack after	Note
`a`, `b`	q_{ab}	no	$[q_0, q_a]$	
`c`	q_{abc}	yes (`abc`)	$[q_0, q_a, q_{ab}]$	
`a`	q_{abca}	no	$[q_{abc}]$	cleared on leaving q_{abc}
`b`	q_{abcab}	no	$[q_{abc}, q_{abca}]$	
`x`	dead			stuck

Phase 2 pops q_{abca} and unreads `b`, then pops q_{abc}, which accepts, and unreads `a`. It returns `abc`, and the next token starts at `abx`. Three characters (`a`, `b`, `x`) were read past the token and given back.

Technique: Counting reads and pushbacks

- A **read** is every character fetched, including the one that gets the scanner stuck. Re-reading a pushed-back character counts again. End of file is usually not counted; say so explicitly.
- **Pushback** for one token is the number of characters read after the end of the chosen lexeme.
- **Maximum pushback** of a specification is $1 + \ell$, where ℓ is the longest path through *non-accepting* DFA states that starts right after an accepting state. In the IF/ID/NUM/FLOAT DFA that path is G→I→J, so the maximum is 3, as in `3e-y`. If such a path can loop through a *cycle* of non-accepting states, the pushback is unbounded, and that is the warning sign for the next box.

Worked example: The quadratic trap, and Reps' fix

With rules `abc` and `(abc)*d` on $(abc)^k$ with no `d` anywhere, every token reads to the end of the input hoping for a `d`, then backs up to its own `abc`. That is k tokens times $O(n)$, so $\Theta(n^2)$. For a count, take rules `ab` and `(ab)*c` on $(ab)^k$. Token j reads $2(k-j+1)$ characters, so the total is $\sum_j 2(k-j+1) = k(k+1)$ reads, and one token pushes back up to $n-2$ characters.
Reps' fix (TOPLAS 1998): during Phase 2, record every non-accepting configuration (q, i) in a table $hopeless[q, i]$. In Phase 1, stop as soon as (q, i) is hopeless. The output is unchanged, because those reads could only have ended in the same backtrack. Each (q, i) is now explored at most once, so scanning is linear. On $(ab)^k$, every token after the first stops after 3 reads, for $5k-4$ reads in total (checked by simulation).

15 Costs and Alternatives

Trade-offs: Four ways to run a regular expression

Approach	Time per input of length n	Memory	Used by
Backtracking over the NFA	exponential worst case, e.g. $(a \mid a)^*b$ on a^n	small	Java, Perl, Python `re`
NFA simulation (Method 2)	$O(n \cdot m)$ for a pattern of length m	$O(m)$	Thompson's original, RE2
DFA (Methods 3–4)	$O(n)$	up to 2^m states, built once	`flex`, JFlex, compilers
Derivatives (Method 6)	builds DFA states on demand	cache of expressions	functional regex libraries

A compiler's token rules are fixed and its inputs are long, so lexers precompute the DFA. One-off searches build DFA states *lazily* (NFA simulation with a cache), as `grep` and RE2 do. That gets the speed of a DFA without the blow-up up front. In practice lexer patterns rarely blow up: for a word-count specification, `flex -v` reports 15 NFA states and only 7 DFA states.

16 Beyond Regular: Lexer States and Quirky Languages

Lexer states (Lecture 2) give the lexer modes: each mode is a separate rule set. In JFlex, `yybegin(S)` switches to mode `S`, and `YYINITIAL` is the default; in `flex` the same idea is `%x S` with `BEGIN(S)`.

```
<YYINITIAL> {
  "/""*"   { yybegin(COMMENT); }
  "if"     { return new Token(TokenType.IF); }
  ...
}
<COMMENT> {
  "*/"     { yybegin(YYINITIAL); }
  [^]      { /* ignore anything, even newlines */
     }
}
```

Trace of `x/*hi*/.`:
- `x`: ID, in YYINITIAL.
- `/*`: switch to COMMENT; no token.
- `h`, `i`: ignored one by one. Inside COMMENT, `if` would not be a keyword.
- `*/`: back to YYINITIAL; no token.
- `.`: DOT.

Nested comments (`/* outer /* inner */ still outer */`) break this. The first `*/` ends the comment, and `still outer */` is lexed as code. The fix is a **depth counter**: increment on `/*` inside

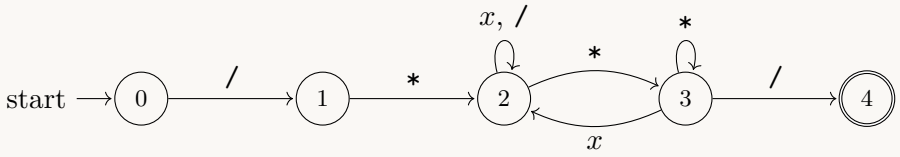
COMMENT, decrement on `*/`, and leave only at depth 0. The counter is extra memory, because nesting is not regular (Method 7).

Worked example: A single regular expression, and its DFA, for `/* ... */` comments

Lexer states are the practical way to skip comments, but a non-nested comment is a *regular* language, and exams ask for the regex and the DFA directly. Let x stand for any character other than `*` and `/`. The difficulty is the body: it may contain `*` and `/` freely, as long as `*/` never appears inside it. Group the body into chunks that can never complete a `*/`: a lone x, a lone `/`, or a run of one or more `*` followed by an x (a `*` followed by `/` would close the comment, and a `*` followed by another `*` is just a longer run). The closing is one or more `*` followed by `/`, since the body may end with extra stars, as in `/* a **/`.

$$/\ *\ (x \mid /\ \mid\ *^{+}x)^{*}\ *^{+}\ /$$

The DFA has five live states, each with a meaning:



State 0: nothing read. State 1: saw `/`. State 2: inside the body, and the last character was *not* a star, so a `/` now is harmless. State 3: inside the body, and the last character *was* a star, so a `/` now closes the comment and another `*` keeps us here. State 4: complete comment, accepting. Every missing edge (for instance x from state 0 or 1) goes to the dead state. The DFA is minimal: states 2 and 3 are distinguished by the suffix `/`, states 0 and 1 by the suffix `**/`, and 4 is the only accepting state. In a real lexer the division operator `/` also starts from state 0, and longest match plus rule priority decide between `/` and a comment, exactly as for every other overlap (§14).
Read the regex and the DFA together: the loop on state 2 is the $(x \mid /)$ part, the trip $2 \to 3 \to 2$ is $*^{+}x$, and the exit $2 \to 3 \to 4$ is $*^{+}/$. Nested comments cannot be added to this picture: a DFA cannot count how many `/*` it has passed (Method 7, §12).

Python: indentation is syntax. Blocks are marked only by indentation, so the lexer invents `INDENT` and `DEDENT` tokens that act like braces.

```
if w == z:
    a = b
else:
    e = f
g = h
```

Python's tokenizer produces: `if w == z :  NEWLINE INDENT a = b NEWLINE DEDENT else :  NEWLINE INDENT e = f NEWLINE DEDENT g = h NEWLINE`.

Algorithm. Keep a stack of widths, starting at [0], and skip blank and comment-only lines. If a line's indent w is greater than the top, push w and emit `INDENT`. If w is smaller, pop and emit `DEDENT` until the top is at most w, and report an error if it is not then equal to w. At end of file, emit one `DEDENT` per entry above 0.

For example, indents 0, 4, 8, 0 give `INDENT`, `INDENT`, then `DEDENT DEDENT` on the last line. Indents 0, 4, 8, 2 pop 8 and 4, and then $0 \neq 2$ is an inconsistent dedent. Each `DEDENT` pops something an earlier `INDENT` pushed, so the work is $O(1)$ amortised per line. No finite automaton can do this. A machine with N states cannot tell $N + 1$ different indentation depths apart, and those depths need different numbers of `DEDENT`s (pigeonhole, Method 7).

Fortran: blanks do not matter. `DO 5 I = 1,25` is a loop, but `DO5I = 1.25` assigns to the variable `DO5I`. Without blanks, both lines start `DO5I=1`, and they differ only at the comma versus

the point. The lexer needs **lookahead beyond the token**. In `flex` this is the trailing-context rule `DO/[A-Z0-9]*=[A-Z0-9]*,`, which matches `DO` only if a comma follows the `=`.

PL/I: keywords are not reserved. In `IF THEN THEN THEN = ELSE; ELSE ELSE = IF`, the keywords are the first `IF`, the second `THEN` and the first `ELSE`; the other words are variables. Only the parser knows a word's role, so the lexer returns a generic identifier, or the parser tells it what to expect. C has a milder version: in `T * x;`, `T` may be a typedef name (a declaration) or a variable (a multiplication). The "lexer hack" lets the lexer consult the symbol table.

17 Lexers in Practice: JFlex, flex, and by Hand

JFlex (Lecture 2, Java) and **flex** (Lecture 4, C) take a specification of rules (pattern plus action) and generate a lexer. Internally they build the combined NFA, convert it to a DFA and minimise it. They scan with longest match, then rule order.

```
Whitespace = [ \t\f\r\n]
Letter     = [a-zA-Z]
Digit      = [0-9]
Identifier = {Letter}({Digit}|{Letter}|_)*
Integer    = "0" | "-"?[1-9]{Digit}*
Float      = {Digit}+ "." {Digit}+
%%
{Whitespace} { /* ignore */ }
"if"         { return new Token(TokenType.IF); }
{Identifier} { return new Token(TokenType.ID,
    yytext()); }
{Integer}    { return new Token(TokenType.INT,
                   Integer.parseInt(yytext())); }
```

```
%{
  /* C declarations, copied verbatim */
%}
digit    [0-9]
id       [A-Za-z]([A-Za-z]|{digit})*
%%
[ \t\n]+  { /* skip */ }
if        { return IF; }
{id}      { yylval = installID(); return ID; }
"<="      { yylval = LE; return RELOP; }
%%
/* user code, copied verbatim, e.g. main() */
```

- `yytext` (or `yytext()` in JFlex) is the matched lexeme, and `yyleng` its length.
- `yylval` carries the attribute, and `yylloc` the position.
- `yylex()` returns the next token to the parser.
- In `flex`, an unmatched character is **echoed by a hidden default rule**. Add a catch-all, or `%option nodefault`.
- `.` excludes newline, but `[^x]` includes it.
- A `flex` pattern must start in column 1.
- Positions can be kept in the macro `YY_USER_ACTION`, which runs before every action and can scan `yytext` for newlines.

Worked example: Predict the output: rule choice in practice

Rules, in order: `a+b` prints `[X:..]`, `ab*` prints `[Y:..]`, `b` prints `[Z]`, `ab` prints `[W]`, and `.|\n` echoes. On `aaabbab` plus a newline, then `abbbca`, `flex` prints `[X:aaab][Z][X:ab]`, a newline, then `[Y:abbb]c[Y:a]`.

- `aaab`: X matches 4 characters, Y only 1.
- `ab`: X, Y and W all match 2, and X is listed first.
- `abbb`: Y matches 4, beating X's 2.

`flex` also warns "rule cannot be matched" for `ab`, since `a+b` always ties with it and comes first.

A hand-written scanner is a DFA in disguise (Lecture 4, Dragon-book code). `ScanOneToken` reads a character, skips blanks, and `switch`es on it:

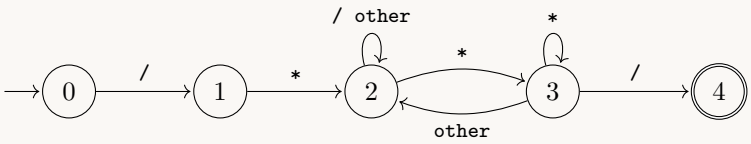
- the `switch` is the DFA's start state, with one `case` per transition;
- a `for`/`while` loop on `isupper`, `islower` or `isdigit` is a self-loop, so it implements longest match;
- `ungetc(ch, fp)` pushes back the character that ended the loop;
- `default:` returning `T_UNKNOWN` is the catch-all.

Single-character tokens use their ASCII code as the token number, and other tokens start at 257, above every character code. The token is a `struct` with an `int type` and a `union` attribute, which holds a string, an `int` or a `double`. Keywords are found by lookup in a reserved-word table. The driver is `while (ScanOneToken(stdin, &token) != T_END)`.

The slide's code has a real bug, checked by running it. After `//`, the placeholder "here you would skip the comment" falls through and returns `'/'`, and then the comment text is scanned as code: `// halve` gives `'/' ID(halve)`. Note also that `ungetc` guarantees only *one* character of pushback. That is enough here, but not for `3e-y`.

Worked example: Deriving the block-comment regex from its DFA

Do not guess; draw the DFA. For `/* ... */` over the classes `/`, `*` and *other* (which includes newline):



State 2 means "inside, the last character was not `*`". State 3 means "inside, just read one or more `*`". With the dead state there are 6 states, and the DFA is minimal. Inside the comment, the machine either stays in 2 on `[^*]`, or makes the round trip 2→3→2 on `"*"+[^*/]`. It leaves on `"*"+"/"`. Hence

```
"/*" ( [^*] | "*"+ [^*/] )* "*"+ "/"
```

- It accepts `/**/`, `/***/` and `/* a * b **/`.
- It rejects `/*/`.
- On `/* a */ b */` it matches only `/* a */`.

The tempting rule `"/*".*"*/"` is wrong twice. On `x /* c1 */ y /* c2 */ z`, longest match stretches `.*` to the *last* `*/` on the line, and `y` vanishes. And since `.` cannot cross a newline, multi-line comments never match. Rule order cannot fix this, because the problem is the length.

Part IV

One Complete Worked Example

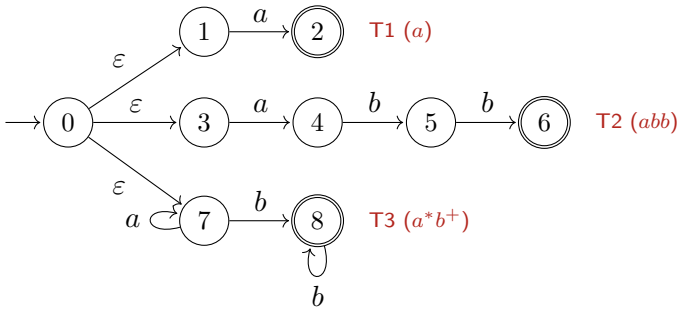
18 From a Specification to a Running Lexer

This is the Dragon book's classic lexer, done end to end. The alphabet is $\{a, b\}$, and there are three rules in priority order:

Rule	Pattern	Token
1	a	T1
2	abb	T2
3	a^*b^+	T3

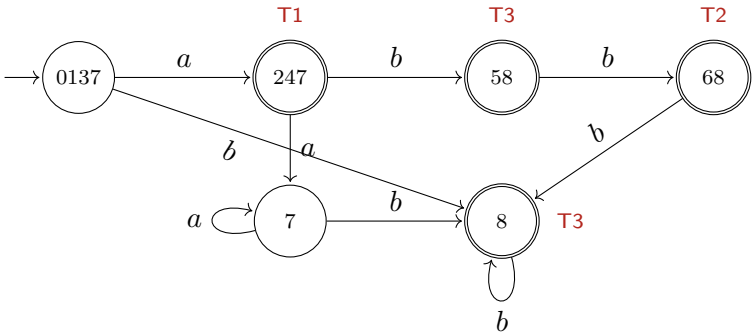
The rules overlap on purpose. `a` is T1 only. `abb` matches both T2 and T3, so priority must decide. And `aab` is T3 only, yet it starts with a T1 match.

Step 1: one NFA per rule, merged. To keep the picture readable we use small hand-built NFAs instead of the full Thompson NFAs. Thompson's would have more ε-states, but after Steps 2 and 3 they give the same DFA. A new start state 0 gets ε-edges to each rule's start.



Step 2: the subset construction, with labels. The start state is ε-*closure*$(\{0\}) = \{0, 1, 3, 7\}$.

DFA state	on a	on b	Accepting NFA states	Label
$\{0,1,3,7\}$ (start)	$\{2,4,7\}$	$\{8\}$	none	—
$\{2,4,7\}$	$\{7\}$	$\{5,8\}$	2 (T1)	**T1**
$\{8\}$	—	$\{8\}$	8 (T3)	**T3**
$\{7\}$	$\{7\}$	$\{8\}$	none	—
$\{5,8\}$	—	$\{6,8\}$	8 (T3)	**T3**
$\{6,8\}$	—	$\{8\}$	6 (T2), 8 (T3)	**T2** (rule 2 before rule 3)



Step 3: minimise, respecting labels. For a lexer, states with different labels must never merge. So the initial partition separates the labels: {non-accepting: 0137, 7}, {T1: 247}, {T3: 8, 58}, {T2: 68}.

- On a, 0137 goes to 247 (T1) while 7 goes to 7 (non-accepting), so they split.
- On b, 8 goes to 8 (T3) while 58 goes to 68 (T2), so they split.

Every block is now a single state: the DFA is already minimal, with 6 states.

Step 4: the table *is* the lexer.

State	a	b	Token if the scan ends here
0137	247	8	—
247	7	58	T1
7	7	8	—
8	dead	8	T3
58	dead	68	T3
68	dead	8	T2

Step 5: scan `abbbabbaaa` with longest match (§14).

Token	Reads (states visited)	Last accepting position	Emit, push back
1	a:247(T1), b:58(T3), b:68(T2), b:8(T3), a:stuck	after 4 characters, T3	**T3** `abbb`, push back a
2	a:247, b:58, b:68(T2), a:stuck	after 3, T2	**T2** `abb`, push back a
3	a:247(T1), a:7, a:7, end of input	after 1, T1	**T1** `a`, push back aa
4	a:247(T1), a:7, end of input	after 1, T1	**T1** `a`, push back a
5	a:247(T1), end of input	after 1, T1	**T1** `a`

The result is **T3 T2 T1 T1 T1**, after 15 reads for 10 characters (checked by simulation). Each token shows one lesson:

- **Token 1**: after 3 characters the best label was T2, but the scan continued, and T3 with 4 characters won. Longest match comes before priority.
- **Token 2**: `abb` ties between T2 and T3, and priority picks T2.
- **Token 3**: the scanner wandered through the non-accepting state 7 hoping for a b (an a^*b^+ match), then had to give back two characters. This is the kind of read-ahead that turns quadratic on long inputs like a^n, which Reps' fix cures.

Part V

Exam Kit

19 Checklists, Formulas and Traps

Technique: Method checklists

- **Thompson:** parse first; build inside out; state your convention; check the counts and (P1) to (P3).
- **Simulation and subset construction:** take the closure after every move; write sets sorted; a set is accepting if it contains an accepting NFA state; stop when no new set appears; the empty set is the dead state.
- **Minimisation:** list all pairs; Rule 1 first; sweep Rule 2 using marks from earlier sweeps; merge unmarked pairs; *relabel all incoming transitions*; for lexers, start by separating different labels.
- **State elimination:** add a new final state f; to remove q, use AL^*B for each in/out pair; join parallel edges with |.
- **Lower bound or non-regular:** exhibit pairwise distinguishable strings, each pair with its

suffix; infinitely many of them means not regular.
- **Scanning:** longest match first, priority second, catch-all last; count every read including the stuck character; push back everything after the last accept.

Fact	Formula or value
Thompson states, convention A	$2(n_\ell + n_\mid + n_*)$; concatenation adds one ε-edge
Thompson states, convention B	$2(n_\ell + n_\mid + n_*) - n_\cdot = 2(\#\text{ops} + \#\text{operands}) - 3n_\cdot$
Thompson edges	B: $n_\ell + 4(n_\mid + n_*)$; A: add $n_\cdot$. At most 2 out-edges per state
NFA simulation	$O(\lvert S\rvert \lvert Q\rvert^2)$ in general; $O(\lvert S\rvert \lvert Q\rvert)$ for Thompson NFAs
DFA scan	$O(\lvert S\rvert)$; up to $2^{\lvert Q\rvert}$ states; L_k needs 2^k
Minimisation	table filling $O(n^2)$ pairs; Hopcroft $O(n \log n)$; the minimal DFA is unique
Maximum pushback	$1 +$ the longest non-accepting run after an accepting state
Quadratic trap	$\Theta(n^2)$ reads, e.g. $k(k+1)$ for `ab`, `(ab)*c` on $(ab)^k$; Reps' fix makes it linear

Warning / Gotcha: Classic traps

- One failed NFA path does not reject. Reject only when *all* paths fail, or when the subset ends without an accepting state.
- After merging DFA states, relabel arrows that point *into* merged states, from anywhere.
- Rule order never overrides length: `infinity`, `classroom` and `ending` are identifiers.
- `read()` without pushback loses a character, as in `sum+1`. Committing too early breaks `x==y`.
- Tokens are what the specification says: `-3.5` is two tokens under Lecture 2's rules, and a sign inside a number pattern breaks `x-1`.
- `.*` in a comment rule overruns to the last `*/`; `.` excludes newline, while `[^x]` includes it.
- The two Thompson conventions give different counts: (0|1)*1 has 10 or 9 states. State yours.
- Nested comments, balanced parentheses and Python indentation are not regular, and need a counter or a stack.

Problem set question	Where it is prepared
PS2.1 classify tokens; PS2.2 positions	§1; §2 (position counting)
PS2.3 the string-reading bug	§2 (consume versus peek; append the quote only after the test)
PS2.4 hex literals; PS2.5 JFlex spec and `end`/`ending`	§3; §4; §17
PS3.1 NFA traces, subset construction and blow-up	§5; Method 3 (§8)
PS3.2 Thompson on $(a \mid b)^*c$	Method 1 (same shape as $(0 \mid 1)^*1$)
PS3.3 subset construction with ε	Methods 2–3
PS3.4 minimisation; PS3.5 `class`/`classroom`	Method 4 (§9); §4
PS4.1 comment regex, numbers	§17 (comment DFA); §3
PS4.2 Thompson size and cost	Method 1; Proposition 7.1
PS4.3 munch, pushback, quadratic trap	§14
PS4.4 exponential DFAs; PS4.5 indentation and Fortran	Method 7 (§12); §16
PS4.6 `flex`	§17

Summary: The whole block in ten lines

1. A lexer cuts characters into tokens $\langle$name, attribute$\rangle$ with positions; the parser pulls them one at a time.
2. Hand-written lexers fail on lookahead; specify tokens declaratively with regular expressions instead.
3. Regex core: a, ε, $|$, concatenation, *. Everything else is sugar. Abbreviations are never recursive.
4. Conflicts: longest match first, rule order second, catch-all last.
5. Thompson: two states per leaf, union and star; concatenation is free (A: an ε-edge; B: a merge).
6. Simulation and subset construction: $\delta(S, c) = \varepsilon\text{-}closure(move(S, c))$. DFAs may be exponential, and that is real.
7. Minimisation: Rule 1, Rule 2 sweeps, merge, relabel. The minimal DFA is unique (Hopcroft: $O(n \log n)$).
8. The way back: state elimination gives a regex; derivatives give a DFA directly; distinguishable strings give lower bounds.
9. The lexer: a merged NFA with labelled accepts, a DFA labelled by priority, scan and backtrack, and Reps' memo against the quadratic case.
10. Real languages need more: lexer states (comments), a stack (Python), lookahead (Fortran) or the parser (PL/I, C).